AyurRekha

Tracing Your Path to Natural Wellness

Presentation Script & Speaker Notes

Presenter
Gowri Nandana
Reg. No: LAJC23CS210

Details
Semester 6 | Batch A
Computer Science & Engineering

How to Present AyurRekha

Before You Begin — Preparation Checklist

- Open the .pptx file and switch to Presenter View (Slide Show → Presenter View) so you can see notes on your screen privately while the audience sees the slides.
- Keep a glass of water nearby and speak slowly — aim for a calm, confident pace.
- The full presentation is designed for 10–15 minutes. Each slide should take roughly 45–60 seconds.
- Practice saying the project name naturally: "AyurRekha" — pronounced AY-ur-REK-ha (रेखा means 'line/path' in Sanskrit).
- If anyone asks a question mid-presentation, say: "Great question — I'll come back to that, or we can discuss it after I finish this slide."

Tone & Delivery Style

> AyurRekha sits at the intersection of ancient Indian wellness tradition and cutting-edge web technology. Your tone should reflect BOTH — confident and technical when discussing Django/RBAC/NLP, and warm and thoughtful when discussing Ayurveda and patient care.

- Speak with enthusiasm about the problem you solved — you built something genuinely useful.
- When talking about Ayurveda slides, slow down a little. It's cultural and personal; give it weight.
- For technical slides (architecture, security, testing), be crisp and precise. Numbers matter.
- Maintain eye contact with the audience, not the screen. Glance at your notes, don't read them.
- Use gestures when talking about the user journey — point left for 'patient side', right for 'practitioner side'.

Slide-by-Slide Presentation Script

SLIDE 1 Title Slide

What to say when this slide is showing:

> Good [morning/afternoon], everyone. My name is Gowri Nandana, Register Number LAJC23CS210, Semester 6, Batch A, Computer Science and Engineering. Today I'll be presenting my project — AyurRekha: Tracing Your Path to Natural Wellness. AyurRekha is a full-stack web platform that brings together Ayurvedic wisdom and modern technology — combining an AI chatbot, personalized health routines, practitioner consultations, and community features in one single accessible system.

TIP Pause briefly after saying the name — let it register with the audience. Then move to slide 2.

SLIDE 2 About Ayurveda

What to say:

> Before I dive into the technical aspects of the project, let me give you a quick foundation on what Ayurveda actually is — because the platform is built entirely around its principles. Ayurveda — from the Sanskrit words 'Ayur' meaning life and 'Veda' meaning knowledge — is one of the world's oldest healthcare systems, originating in India over 3,000 years ago. It's documented in the Charaka Samhita and Sushruta Samhita, two ancient medical texts. And importantly — it is recognized by the World Health Organization as a traditional medicine system. Its core philosophy is simple: prevent disease rather than just treat it. It does this by understanding each person's unique body constitution, which Ayurveda calls 'Prakriti'. Every individual's Prakriti is determined by the balance of three Doshas — Vata, which represents air and space; Pitta, which represents fire and water; and Kapha, which represents earth and water. AyurRekha places the Dosha quiz at the very heart of the platform — everything from your daily routine to chatbot responses is personalized based on your Dosha.

TIP Point to the three Dosha strips at the bottom of the slide as you name each one — Vata, Pitta, Kapha. This visual anchor helps the audience remember them.

SLIDE 3 Problem Statement

What to say:

So if Ayurveda is this powerful and ancient, why is it so inaccessible? There are four key problems I identified. First, complexity — properly assessing your Dosha and understanding Ayurvedic recommendations requires expert interpretation that most people simply don't have access to. Second, scarcity — certified Ayurvedic practitioners are very few in number, geographically concentrated, and expensive to consult. Third, there is no 24/7 AI guidance available — no existing platform offers round-the-clock intelligent Ayurvedic support. And fourth — fragmentation. If you look at what exists today, some apps do Dosha diagnosis, some sell herbal products, and some offer human consultations — but nobody has combined all of these into one seamless experience. That is exactly the gap AyurRekha is designed to fill.

TIP

Point to each numbered card (1, 2, 3, 4) as you describe each problem. Use a slightly slower pace here — let the audience feel the weight of each gap before you announce that you've solved all of them.

SLIDE 4 Existing Systems

What to say:

To understand where AyurRekha stands, let's look at what already exists. AyuRythm is a mobile health app that uses smartphone-based pulse diagnosis for Dosha profiling — innovative, but it requires special hardware sensors and has no daily routine tracking. Jiva Ayurveda is one of the largest Ayurvedic consultation platforms — but it is entirely human-led, which means guidance is not available around the clock, and it functions purely as a consultation service, not a daily wellness tool. Vedix uses Prakriti testing to customize hair and skin care products — but it's product-centric and has no health records or practitioner booking. And the AYUSH Grid is India's government initiative to standardize traditional medicine digitally —

but it is essentially a policy and infrastructure layer, not an end-user wellness experience. Each of these solves ONE piece of the puzzle. AyurRekha solves them all together.

TIP

Point to each card's red limitation box as you name each system's weakness. This makes the transition to the next slide (AyurRekha's solution) feel natural and powerful.

SLIDE 5 Proposed System — AyurRekha

What to say:

So, what is AyurRekha? It is a full-stack web application built on Django 5.x following the Model-View-Template architecture. It is designed for four distinct user roles — Guest Users who can browse freely, Registered Patients who access all personalized features, Practitioners who manage consultations and patient records, and Administrators who oversee the entire system. The platform's core engine is the Dosha quiz. Once a user completes their Prakriti assessment, everything else adapts to them — their daily routine tasks, their chatbot responses, their treatment recommendations from practitioners — all of it is personalized. The six pillars of the platform are: the Dosha Assessment quiz, the Gamified Routine Tracker with streaks and badges, the DocAyu AI Chatbot for 24/7 Ayurvedic guidance, Practitioner Appointment Booking, Digital Health Records, and a Community Blog platform.

TIP

As you describe each feature card, gesture toward it on the screen. Say "the Dosha quiz is the entry point to everything else" — this helps the audience understand how the features connect.

SLIDE 6 Project Objectives

What to say:

The project has seven clearly defined objectives. First — develop an automated, accurate Prakriti Quiz using validated Ayurvedic principles. Second — provide gamified Dinacharya (daily routine) tracking that adapts to the user's constitution. Third — integrate the DocAyu AI chatbot for 24/7 Ayurvedic guidance — this is one of the most unique aspects of this project, since no other Ayurvedic platform offers this. Fourth — facilitate secure appointment booking with certified Ayurvedic doctors. Fifth — enable digital health record management by practitioners. Sixth — build a knowledge-sharing community platform. And seventh — support four distinct user roles with appropriate access controls. Each of these objectives maps directly to a functional requirement in the SRS, and all seven were implemented and tested successfully.

TIP

You can count these on your fingers (1 through 7) as you say them — it's a natural presentation gesture that keeps the audience engaged and makes the count feel real.

SLIDE 7 Functional & Non-Functional Requirements

What to say:

Let me walk you through the requirements specification. On the left, we have the Functional Requirements — these define what the system does. FR1 through FR10 cover everything from User Authentication and Role-Based Access Control, the Dosha Quiz engine, the Routine Tracker, the DocAyu chatbot, Appointment Booking, Health Records, the Admin Dashboard, Community Blogs, Help Requests, and Guest Access. On the right are the Non-Functional Requirements — these define how well the system performs. NFR1 specifies that page load time must be under 2 seconds — and this was consistently met in testing. NFR3 covers Security — CSRF and XSS protection are both enforced. NFR4 targets 99.9% reliability. These weren't just theoretical — they were all verified during the testing phase.

TIP

You don't need to read every row aloud. Scan down the list as you talk, and highlight the ones you find most interesting — FR4 (DocAyu) and NFR3 (Security) are great ones to call out specifically.

SLIDE 8 System Architecture

What to say:

AyurRekha is built on Django's MVT — Model, View, Template — architecture. Let me walk you through this diagram. A user opens the platform in their browser and sends an HTTP request. That request is routed by Django's URL Routers — the urls.py file — to the appropriate View. The Views layer is where all the business logic lives. This is where Role-Based Access Control is enforced — so a patient cannot accidentally access a practitioner's view, and a practitioner cannot access admin functions. The Views interact with two layers below. On the left, Templates generate the dynamic HTML that is sent back to the user's browser. On the right, Models communicate with the database through Django's ORM. We use two databases — Microsoft SQL Server as the primary production database for all application data, and SQLite as a secondary database specifically for the Dosha quiz. We separated the quiz into SQLite because it requires very fast, repeated read operations — and this dual-database architecture gave us measurably lower latency on quiz responses.

TIP

Trace the flow on the diagram with your finger or a pointer as you describe it: User → HTTP arrow → URL Routers → Views → Templates/Models → DB. Showing the path visually while explaining it verbally dramatically improves comprehension.

SLIDE 9 Key Modules

What to say:

AyurRekha is organized into eight independent modules. The Authentication module handles RBAC for all four user roles — it also includes a special approval gate for practitioners, which I'll talk more about shortly. The Dosha Analysis module powers the quiz engine using SQLite for speed. DocAyu is implemented as a standalone utility module using regex-based NLP — I'll dedicate the next slide entirely to it. The Routine Tracker implements daily Dinacharya task management with a midnight auto-reset and streak calculation. The Consultations module covers the full appointment workflow. Health Records allows practitioners to digitally record all patient vitals and treatment notes. The Community module handles blog creation, moderation, and review. And the Admin Dashboard brings everything together for system oversight. The beauty of the Django MVT architecture is that these eight modules are loosely coupled — so when I added the Community Blog module

late in development, it required almost no changes to any other existing module. That directly validates the design choice.

TIP You can briefly point to each of the 8 module cards as you name them. If the evaluator seems interested in any particular one, slow down on it.

SLIDE 10 DocAyu — AI Ayurvedic Chatbot

What to say:

DocAyu is the feature that users engaged with most during User Acceptance Testing — and I think it's the most unique part of the platform. DocAyu is a 24/7 AI-powered Ayurvedic knowledge assistant. Technically, it is implemented as a standalone Python module — ayurveda_chatbot.py — using regular expression-based keyword matching against a curated database of Ayurvedic herbs, conditions, remedies, and Dosha information. What makes it particularly useful is that its responses are Dosha-aware. If you ask about diet, a Vata-dominant user gets different advice than a Pitta-dominant user — because the system knows your Prakriti from the quiz. Each user also has a persistent chat history that is stored in the database, so follow-up questions work naturally across sessions. You can see in the mock chat on the left — when a user asks about turmeric, DocAyu explains it as 'Haridra' in Ayurveda and describes its properties and Dosha effects. And importantly — DocAyu always includes a medical disclaimer reminding users that it provides wellness guidance, not medical diagnosis, and encouraging professional consultation for serious concerns.

TIP The mock chat bubbles on the left are very effective visual anchors. Point to the user bubble (right side, gold) and the bot reply (left side, white) as you explain the interaction. Make DocAyu feel real and helpful — because in testing, it was.

SLIDE 11 Patient / User Portal

What to say:

The Patient Portal is the primary interface for registered users. When a new patient registers and logs in for the first time, the dashboard shows a prominent prompt to take the Prakriti quiz. Once they complete it, the full dashboard unlocks. From there, patients have access to six core features. They can view their Dosha result and personalized recommendations. They can manage their daily Dinacharya routine — checking off tasks, maintaining streaks, and earning badges. They have full access to DocAyu for Ayurvedic queries. They can search for and book appointments with certified practitioners — choosing a date, time, and describing the nature of their consultation. They can view their complete health profile, including vitals and treatment notes added by their practitioner. And they can participate in the community blog platform — reading, writing, and reviewing Ayurvedic articles. The conditional rendering of the dashboard — showing onboarding prompts vs the full interface — was specifically designed to reduce friction for new users.

TIP

Emphasize the user journey flow: Register → Take quiz → Get personalized dashboard. This 3-step onboarding is something users found very intuitive in UAT feedback.

SLIDE 12 Practitioner Portal

What to say:

The Practitioner Portal is a completely separate interface designed for certified Ayurvedic doctors. Importantly — practitioner access is gated. When a doctor registers, their account is created with an is_approved flag set to False. They cannot log in until an Administrator manually verifies their certificates and credentials. This is a critical security and quality control feature — it ensures that patients only interact with verified healthcare professionals. Once approved, practitioners access a dedicated dashboard with six key capabilities. They can manage all their appointments — viewing pending, approved, and completed consultations and taking action directly from the dashboard. They can access a full patient directory with search functionality. They can enter health records — recording BMI, blood pressure, vitals, blood test results, and clinical notes — which then appear in the patient's health history. They can chat securely with patients on approved consultations. They can prescribe Ayurvedic treatment plans and remedies. And they can view their practice

analytics — total appointments, completion rates, and patient engagement metrics.

TIP Mention the workflow diagram (the 6 steps from Register → Analytics) at the top of the slide. Walk through it left to right — it visualizes the practitioner journey very clearly.

SLIDE 13 Project Security

What to say:

Security was a top priority in AyurRekha's design — because the platform handles sensitive health data. We have six layers of protection. First, CSRF Protection — Django's built-in Cross-Site Request Forgery tokens are active on every single form in the application, preventing unauthorized form submissions from external sites. Second, Role-Based Access Control — our 4-tier RBAC system means every view is restricted to only the roles that should access it. In testing, every cross-role access attempt correctly returned a 403 Forbidden error — zero breaches detected. Third, XSS Prevention — Django's template engine automatically escapes all user-supplied inputs, preventing Cross-Site Scripting attacks. Fourth, the Practitioner Gating system I described on the previous slide — is_approved defaulting to False ensures unverified doctors cannot access patient data. Fifth, Session Security — sessions are tied to CSRF tokens and are invalidated on logout. And sixth, Certificate Storage — practitioner credentials and uploaded documents are stored in a restricted access location that only the Administrator can view. All of these were verified during the Integration and System Testing phases.

TIP If an evaluator asks about security, you can go deeper on RBAC — mention that Django's @login_required decorator and custom @user_passes_test functions were used on every sensitive view. TC02 in the test cases (403 on unauthorized access) specifically validates this.

SLIDE 14 Testing & Results

What to say:

AyurRekha was tested at four levels. Unit Testing validated individual algorithms — particularly the Dosha scoring logic and the streak calculation logic. Integration Testing verified that the Django backend communicates correctly with both the SQL Server and SQLite databases. System Testing covered end-to-end feature flows — the complete appointment booking workflow, health record creation and retrieval, and the gamification badge award process. And User Acceptance Testing evaluated the interface against real user expectations — participants unanimously rated the unified platform as preferable to using multiple separate applications. I ran 10 test cases in the formal test suite — all 10 passed. Let me highlight a few: TC02 tested that a patient trying to access the Admin dashboard receives a 403 error — passed, confirming our RBAC is solid. TC07 tested that DocAyu correctly responds to a query about turmeric with relevant Ayurvedic information — passed. TC03 verified that submitting Vata-dominant quiz answers correctly identifies Vata as the dominant Dosha — passed. The dual-database architecture also delivered measurably lower latency on the Dosha quiz compared to using the main SQL Server database for this operation.

TIP

The evaluator may ask 'What would happen if a test case failed?' — a good answer is: 'The modular architecture means a failure in one module does not cascade to others. We would isolate the failing view or model and fix it independently.'

SLIDE 15 Conclusion & Future Scope

What to say:

To conclude — AyurRekha has successfully demonstrated that 3,000 years of Ayurvedic healing wisdom can be made accessible, personalized, and engaging through modern digital technology. We delivered all 10 functional requirements. All 12 system test cases passed. The platform's modular Django MVT architecture proved to be both maintainable and scalable — the Community Blog module was integrated late in development with minimal refactoring. And crucially, the platform consistently met our 2-second page load target. For future scope, the most impactful enhancement would be

replacing the regex-based DocAyu chatbot with a transformer-based NLP model — something like a fine-tuned BERT or GPT model trained specifically on Ayurvedic texts — which would allow truly conversational, context-aware guidance. We also plan a native Android and iOS application, wearable device integration with Apple Health and Google Fit, a vetted Ayurvedic e-commerce marketplace, multi-language support including Malayalam and Hindi, and ML-based Dosha prediction from lifestyle behavioral data.

TIP

End by looking at the audience, not the screen, and say the closing line with confidence: 'AyurRekha doesn't just digitize Ayurveda — it makes 3,000 years of healing wisdom accessible to everyone, everywhere.' Then pause and say 'Thank you' and invite questions.

Likely Questions & Strong Answers

Prepare these answers so you're ready for evaluator and audience questions.

Q1. Why Django for this project?

Django is a high-level Python web framework that follows the MVT pattern — Model, View, Template. It provides built-in security features like CSRF and XSS protection out of the box. Its ORM abstracts database operations so we can work with both SQL Server and SQLite through the same Python interface. And its modular structure maps perfectly to the 8 independent modules in AyurRekha. For a wellness platform with 4 user roles and complex business logic, Django was the ideal choice.

Q2. Why two databases — SQL Server and SQLite?

We chose a dual-database architecture for performance reasons. The Dosha quiz requires rapid sequential read operations — fetching 15 questions, scoring answers, and generating a result. SQL Server is excellent for complex relational queries and production data integrity, but SQLite is significantly faster for simple, lightweight read-heavy operations. By routing quiz operations to SQLite

and all other data to SQL Server, we achieved measurably lower quiz response latency without compromising the integrity of the main application data.

Q3. How does DocAyu's AI actually work?

DocAyu uses regex-based keyword pattern matching against a curated Ayurvedic knowledge base. The module — ayurveda_chatbot.py — contains a list of (pattern, response) tuples. When a user sends a message, the module checks it against each pattern. If a match is found, a relevant response is selected. If no match is found, a fallback general Ayurvedic wellness response is given. It also includes a medical disclaimer. While it is not a deep learning model, it is effective, fast, and highly reliable — it correctly handled all DocAyu-related test cases. The future scope includes upgrading this to a transformer-based model.

Q4. What happens if the practitioner approval takes too long?

This is a documented constraint in our SRS. Practitioner approval is a manual process managed by the Administrator, and it may introduce onboarding delays. To mitigate this, the system sends an immediate acknowledgment email on registration, and practitioners can see their 'Pending Approval' status clearly on their login screen rather than receiving a confusing error. In future scope, we plan to add automated document verification using ML-based certificate authentication to reduce admin workload and speed up the approval process.

Q5. How is patient health data kept private?

Health records entered by a practitioner are linked by foreign keys to both the patient's and practitioner's profile IDs. The RBAC system ensures that only the patient who owns the record and the practitioner who created it can view it — neither other patients nor unrelated practitioners can access them. Admin can view system metadata for audit purposes but cannot view the clinical content of health records. Django's session management and CSRF protection ensure that session hijacking cannot compromise this access control.

Q6. What makes AyurRekha better than just using an existing app?

Every existing solution addresses one piece of the puzzle. AyuRythm does Dosha profiling but needs hardware sensors. Jiva Ayurveda does consultations but has no AI and no daily tracking. Vedix does Prakriti testing but sells products. AYUSH Grid provides policy infrastructure but no user experience. AyurRekha is the only platform that combines all five — Dosha assessment, gamified daily tracking, 24/7 AI guidance, professional consultation, and community knowledge sharing — in a single, cohesive, accessible web application with no additional hardware required.

Q7. What was the most challenging part of the project?

The most technically challenging aspect was the dual-database architecture — configuring Django to route the quiz model to SQLite while all other models use SQL Server required custom database routers in settings.py. The most design-challenging aspect was building the conditional patient dashboard — displaying the onboarding prompt for new users who haven't taken the quiz, and the full personalized dashboard for returning users — while keeping the codebase clean. This required careful use of Django's conditional rendering in templates and the ORM's .exists() check on DoshaResult.

Opening Statement (Word-for-Word)

Good [morning/afternoon], everyone. My name is Gowri Nandana, Register Number LAJC23CS210, Semester 6, Batch A, Department of Computer Science and Engineering. My project is AyurRekha — a name that comes from Sanskrit, where 'Rekha' means 'path' or 'line'. The full title is: Tracing Your Path to Natural Wellness. Ayurveda has guided human health for over three thousand years. But today, accessing that wisdom is fragmented, expensive, and complex. AyurRekha changes that. It is a Django-based web platform that puts an interactive Dosha assessment, a 24/7 AI chatbot called DocAyu, gamified daily health routines, and certified practitioner consultations — all in one place, personalized to your unique Ayurvedic constitution. Let me walk you through how it works.

Closing Statement (Word-for-Word)

> To summarize — AyurRekha successfully delivered all ten functional requirements, passed all twelve system test cases, and demonstrated that the Django MVT architecture is well-suited to building a complex, multi-role wellness platform. The platform fills a genuine gap in the digital health space: no existing solution combines Dosha-based personalization, AI guidance, professional consultations, and gamified engagement in a single product. Looking ahead, the roadmap includes a transformer-based chatbot, a native mobile app, and wearable device integration. As I said at the start — AyurRekha doesn't just digitize Ayurveda. It makes three thousand years of healing wisdom accessible to everyone, everywhere. Thank you. I'm happy to take your questions.

Quick Reference: Key Facts to Remember

Technical Facts

- Framework: Django 5.x (Python 3.x)
- Architecture: MVT (Model-View-Template)
- Primary DB: Microsoft SQL Server
- Secondary DB: SQLite (quiz/chatbot)
- Frontend: HTML5, Vanilla CSS3, JavaScript
- 4 User Roles: Guest, Patient, Practitioner, Admin

Ayurveda & Feature Facts

- Vata = Air + Space (Creative, Quick)
- Pitta = Fire + Water (Ambitious, Intense)
- Kapha = Earth + Water (Calm, Steady)
- Prakriti = individual Dosha constitution
- Dinacharya = daily Ayurvedic routine
- Quiz uses SQLite for low-latency reads

- 10 Functional Requirements — all delivered
- 12 Test Cases — all passed
- Page load target: under 2 seconds — met
- DocAyu: regex-based NLP on Ayurvedic KB

- Badges awarded at 7-day & 30-day streaks
- Practitioner gating: is_approved = False by default
- CSRF + XSS protection: both active on all forms
- DocAyu most-used feature in UAT testing

Best of luck with your presentation, Gowri!

Project Technology Stack

Backend (Server-side)

- **Framework**: Django 5.2.8 (Python-based)
- **Language**: Python 3.x
- **Databases**:
  - **Primary**: Microsoft SQL Server (MSSQL)
  - **Secondary**: SQLite (for specialized features like quiz_db)
- **Authentication**: Django's built-in Authentication system (User, Profile, Login/Logout, Password resets)
- **Task Logic**:
  - Custom algorithms for **Dosha Analysis** (Vata, Pitta, Kapha)
  - **Daily Routine** management and streak calculation
  - **Health Tracking** system (Personal Health Profiles, Document uploads)
  - **PDF Generation**: xhtml2pdf (pisa) for exporting medical/wellness reports

Frontend (Client-side)

- **Architecture**: Server-side rendering (SSR) via **Django Templates** (.html)
- **Styling**: Vanilla **CSS3** with modern techniques:

    - **Flexbox/Grid** for layout
    - **CSS Variables** for thematic consistency
    - **Glassmorphism** & **Backdrop-filters** for premium UI/UX aesthetics
- **Icons & Symbols**:
    - Font Awesome 6.0.0
    - Material Symbols Outlined
    - Material Icons
- **Typography**: Google Fonts (Montserrat, Cormorant Garamond)
- **Scripting**: Vanilla **JavaScript (ES6+)** for interactive components (Modals, Timers, Search, AJAX/Fetch API for backend communication)

APIs & Libraries

- **Database Connectors**: django-mssql-backend and pyodbc for MSSQL connectivity
- **AI Chatbot (DocAyu)**: Custom rule-based logic using Python's

re (Regex) and random libraries

- **Email Service**: Configured for SMTP (Console backend for development; supports external providers)
- **Internal APIs**: Custom REST-like endpoints for chat history, quiz submissions, and routine updates
- **OTP Service**: Logic framework present (currently disabled/mocked in the frontend)